Basic Materials of Python Basic

Yung Chin, Yen

November 19, 2020

Outline

1 Python 簡介 3
- 1.1 What is Python . . . 3
- 1.2 Python 的誕生 . . . 3
- 1.3 Python 的優勢與應用 . . . 6

2 Python 環境建置 8
- 2.1 下載 python . . . 8
- 2.2 python 的幾種編寫環境 . . . 8
- 2.3 兩種執行模式 . . . 12
- 2.4 安裝 Jupyter Notebook . . . 13

3 變數、資料型態、輸出輸入 15
- 3.1 Hello World . . . 15
- 3.2 變數 . . . 15
- 3.3 資料型態 . . . 16
- 3.4 進階輸出: print . . . 17
- 3.5 輸入 . . . 19
- 3.6 註解 . . . 20

4 運算元與運算式 22
- 4.1 算術運算 . . . 22
- 4.2 關係運算子 . . . 23
- 4.3 邏輯運算子 . . . 23
- 4.4 複合指定運算子 . . . 23
- 4.5 實作練習 . . . 23

5 內建函數 24
- 5.1 基本函數與運算子 . . . 24
- 5.2 字串處理函數 . . . 25

5.3 數值處理函數 34
5.4 作業 34

6 判斷結構 34
6.1 if 34
6.2 if . . . else 35
6.3 if . . . elif . . . else 35
6.4 巢狀 if 36
6.5 實作練習 38
6.6 作業 38

7 迴圈結構 38
7.1 for 38
7.2 while 40
7.3 break 41
7.4 continue 41
7.5 for / while + else 42
7.6 巢狀迴圈 43
7.7 實作練習 44
7.8 作業 44

8 資料型別 44
8.1 字串 string 44
8.2 列表 list 46
8.3 元組 tuple 51
8.4 字典 dict 56
8.5 List v.s. Tuple 59
8.6 集合 set 60
8.7 作業 [set] 63
8.8 資料型別整理 64

9 FUNCTION 65
9.1 What for? 65
9.2 函式 (function) 基本操作 65
9.3 引數 (argument) 的傳遞 67
9.4 模組 (module) 69
9.5 遞迴 (recursive) 70
9.6 實作練習 71

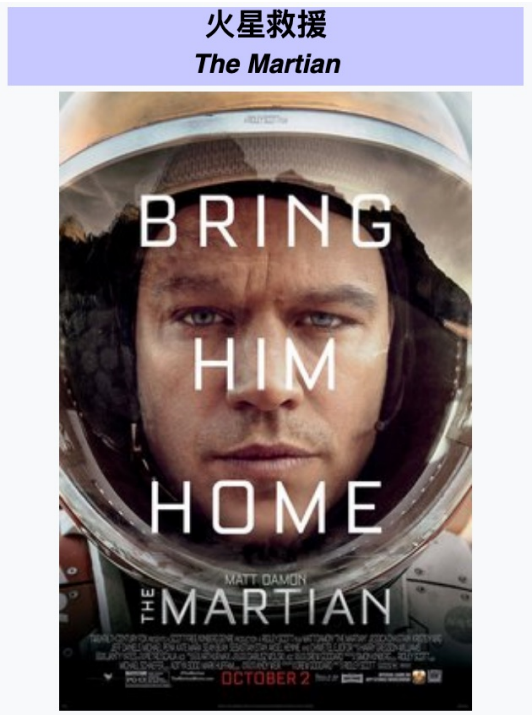
1 Python 簡介

1.1 What is Python

Python is an advanced scripting language that is being used successfully to glue together large software components. It spans multiple platforms, middleware products, and application domains. Python is an object-oriented language with high-level data structures, dynamic typing, and dynamic binding. Python has been around since 1991, and has a very active user community. For more information, see the Python website `http://www.python.org`.

1.2 Python 的誕生

1. 一句話講完一部電影
   - The Martian



   - 一個男人在火星上種菜的故事

2. 一句話講完 Python 的誕生
   - 1989 年，一個宅男工程師為了打發聖誕假期 (順便打趴其他語言)、花了三個月創造了一種新的程式語言。[1]

[1] Python 的誕生故事

- Guido van Rossum

- 30 年後，每天都有數百萬人使用他創立的這一新語言。
  - Python Customers
    Get access to data on 941,453 websites that are Python Customers. We know of
    120,915 live websites using Python and an additional 820,538 sites that used Python historically and 807 websites in Taiwan. [2]
  - 依據 TOIBE 的調查，目前 Python 在TOIBE最受歡迎語言的排名第三[3]

[2]Python Usage Statistics

[3]https://www.tiobe.com/

Apr 2020	Apr 2019	Change	Programming Language	Ratings	Change
1	1		Java	16.73%	+1.69%
2	2		C	16.72%	+2.64%
3	4	^	Python	9.31%	+1.15%
4	3	v	C++	6.78%	-2.06%
5	6	^	C#	4.74%	+1.23%
6	5	v	Visual Basic	4.72%	-1.07%
7	7		JavaScript	2.38%	-0.12%
8	9	^	PHP	2.37%	+0.13%
9	8	v	SQL	2.17%	-0.10%
10	16	^^	R	1.54%	+0.35%
11	19	^^	Swift	1.52%	+0.54%
12	18	^^	Go	1.36%	+0.35%
13	13		Ruby	1.25%	-0.02%
14	10	vv	Assembly language	1.16%	-0.55%
15	22	^^	PL/SQL	1.05%	+0.26%
16	14	v	Perl	0.97%	-0.30%
17	11	vv	Objective-C	0.94%	-0.57%
18	12	vv	MATLAB	0.93%	-0.36%
19	17	v	Classic Visual Basic	0.83%	-0.23%
20	27	^^	Scratch	0.77%	+0.28%

– 幾種不同語言受歡迎程度的變化:TIOBE 2001-2019

- 研究人員於 2019 年首次將 5500 萬光年之外黑洞的照片拼湊出來，使用的程式語言也是 Python。

3. Guido van Rossum 自己的說法
   - ...in December 1989, I was looking for a "hobby" programming project that would keep me occupied during the week around Christmas.[4]

   - My office (a government-run research lab in Amsterdam) would be closed, but I had

[4]Guido van Rossum

a home computer, and not much else on my hands. I decided to write an interpreter for the new Script language I had been thinking about lately: a descendant of ABC that would appeal to Unix/C hackers.

- I chose Python as a working title for the project, being in a slightly irreverent mood (and a big fan of Monty Python's Flying Circus). [5]

1.3 Python 的優勢與應用

1. Advantages[6]
   (a) Easy to learn

   (b) High-level language

   (c) User-friendly data structures

   (d) Dynamically typed language(No need to mention data type based on value assigned, it takes data type)

   (e) Object-oriented language

   (f) Open source and community development

   (g) Presence of third-party modules

   (h) Extensive support libraries(NumPy for numerical calculations, Pandas for data analytics etc)

   (i) Portable and Interactive

   (j) Portable across Operating systems

2. Applications
   (a) GUI based desktop applications(Games, Scientific Applications)

[5] Foreword for "Programming Python" (1st ed.)

[6] Python Language advantages and applications

   (b) Web frameworks and applications

   (c) Enterprise and Business applications

3. Organizations using Python
   (a) Google (Components of Google spider and Search Engine)

   (b) Youtube

   (c) Instagram

   (d) Netflix

   (e) Dropbox

   (f) Reddit

   (g) Uber

   (h) Pinterest

2 Python 環境建置

2.1 下載 python

- google: python download
- 網址: https://www.python.org/downloads/
- 若為 Mac 系統，則已預載安裝 Python 2.7
- 若為 Linux 系統，則多數也預載 Python 2.x 及 3.x
- 查詢目前的 python 版本，目前官方最新版本為 3.8.2[7]

```
python --version
```

2.2 python 的幾種編寫環境

1. 終端機 (shell, command line): interpreter
   (a) 執行環境
      - Linux / Mac: terminal
      - Windows: 命令提示字元

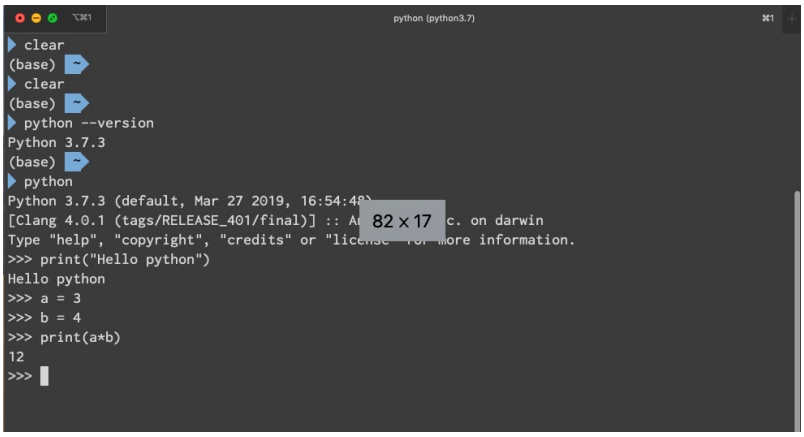


Figure 1: 命令列下執行 Python interpreter

   (b) 執行方式

[7] https://www.python.org/downloads/

- 啟用 Python interpreter

```
python
```

- 結束/回到 terminal
  - 指令

```
exit()
```

  - 快速鍵: Ctrl-D

- 此模式可配合文字編輯器 (如Sublime Text 3、Notepad++) 撰寫程式碼，若檔名為 test.py，執行該程式的方法為：於終端機下輸入

```
python test.py
```

2. IDE(Integrated Development Environment) 環境
   (a) Spyder

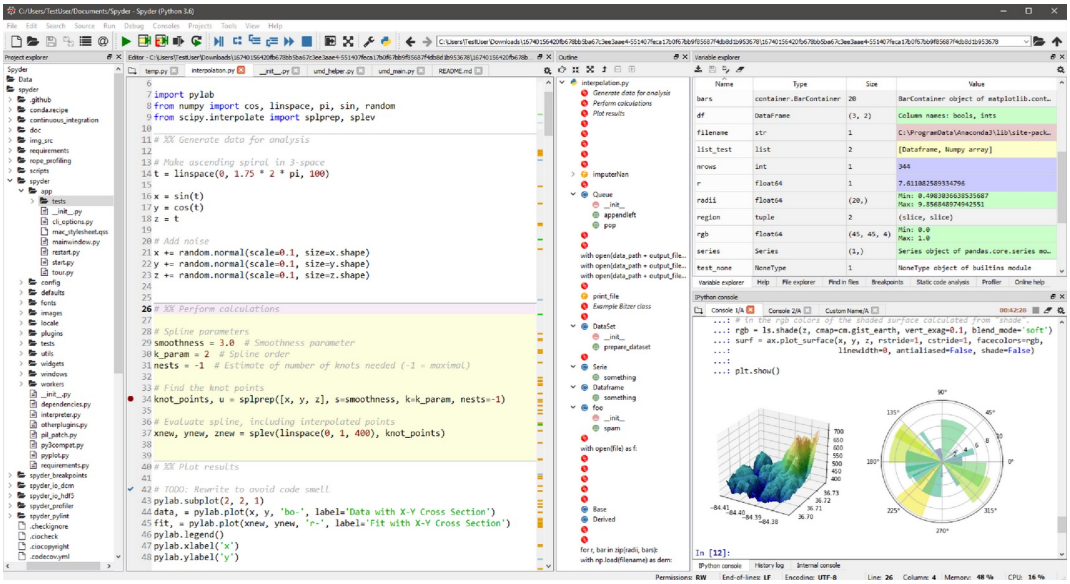


Figure 2: Spyder IDE for Python

   (b) Visual Studio Code
   (c) PyCharm
3. Web-based 環境
   (a) The Jupyter Notebook
   (b) Google Colaboratory
   (c) Python Online Compiler: repl.it

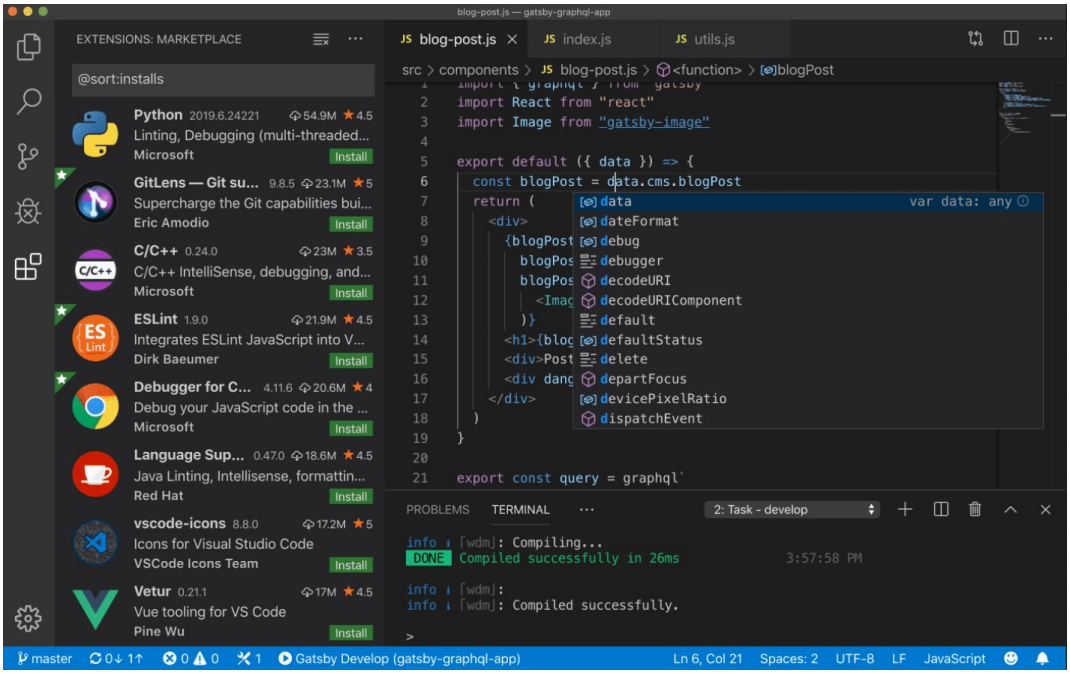


Figure 3: Visual Studio Code

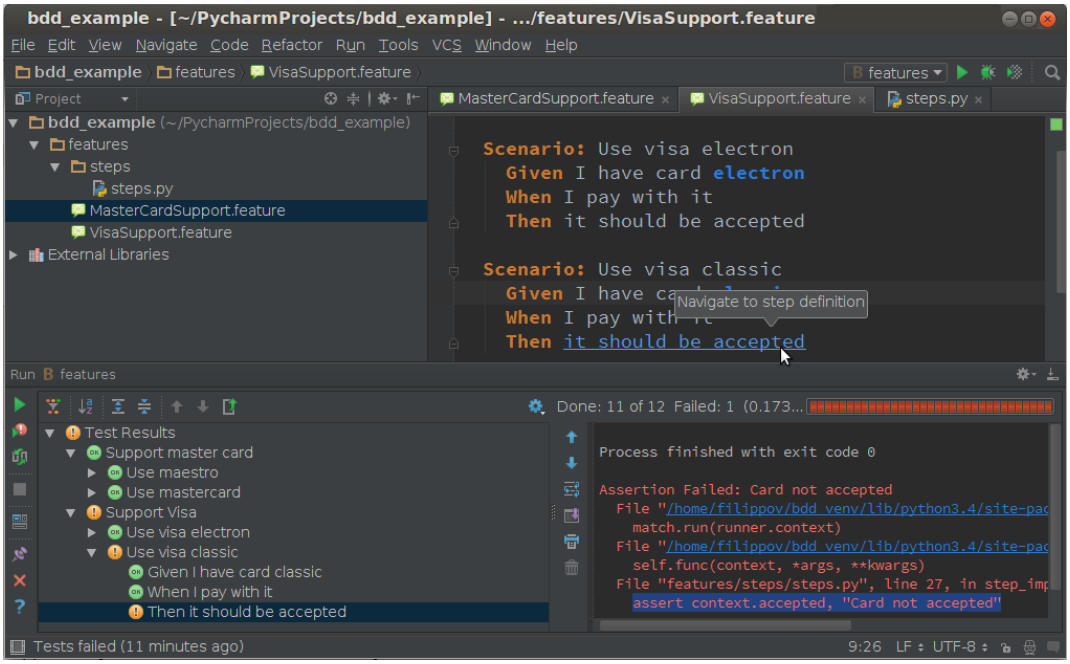


Figure 4: PyCharm

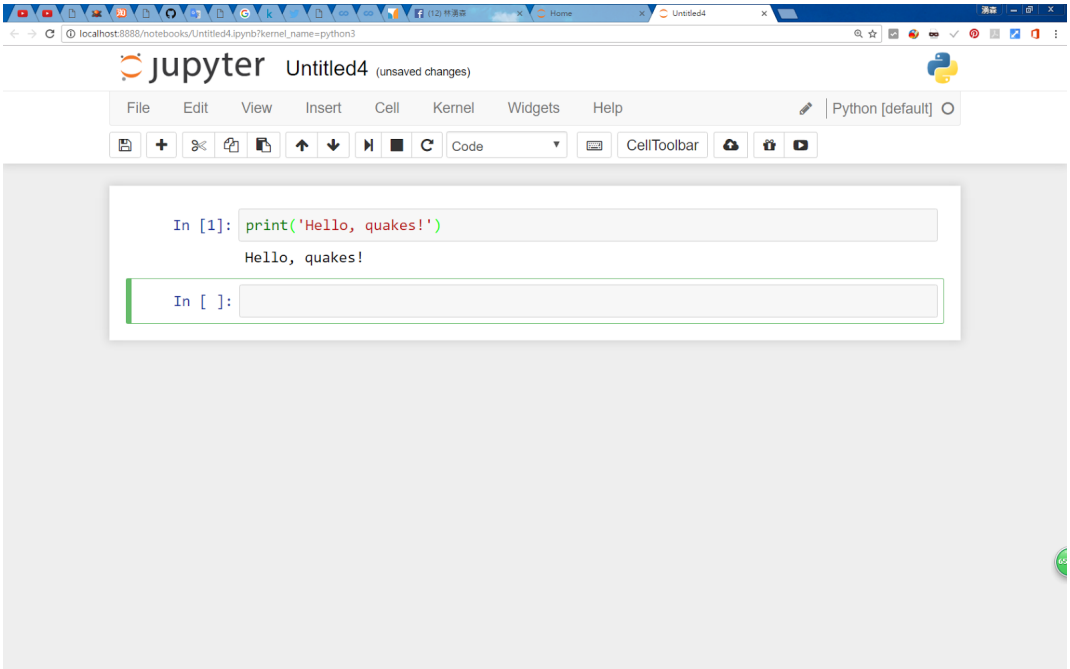


Figure 5: The Jupyter Notebook

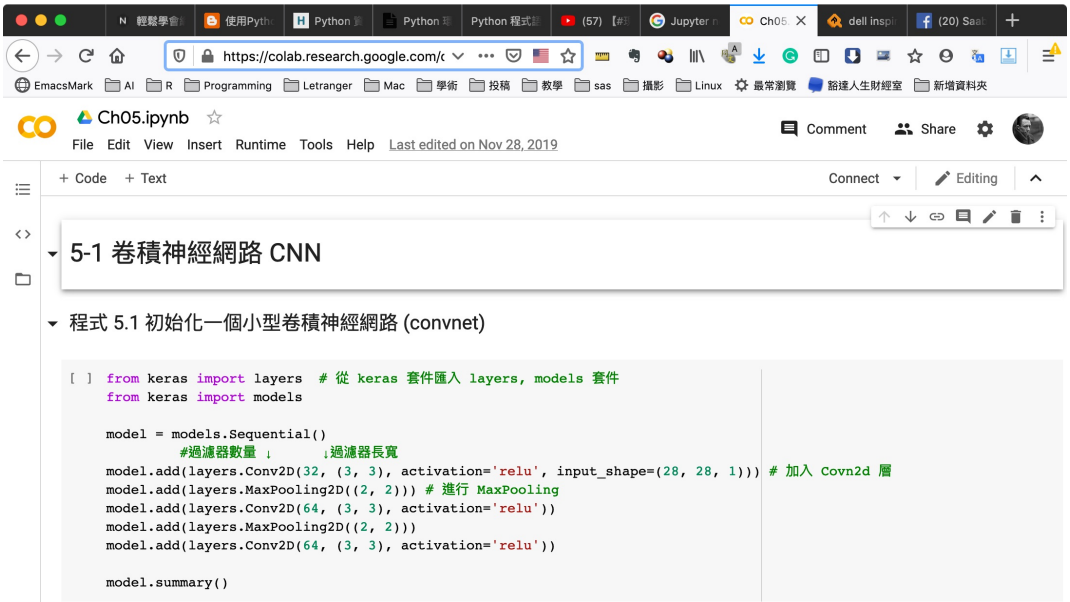


Figure 6: The Google Colaboratory

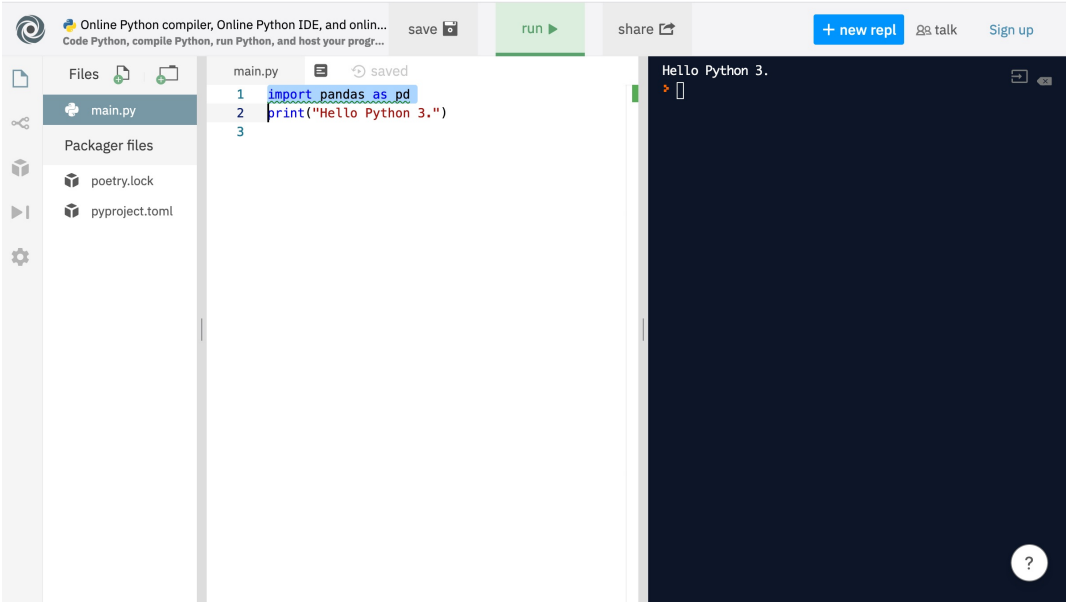


Figure 7: Repl.it

2.3 兩種執行模式

1. Interactive mode
   (a) **啟動方式**

```
python -i filename.py
```

   (b) **優點**[8]
   - Helpful when your script is extremely short and you want immediate results.
   - Faster as you only have to type a command and then press the enter key to get the results.
   - Good for beginners who need to understand Python basics.

   (c) **缺點**
   - Editing the code in interactive mode is hard as you have to move back to the previous commands or else you have to rewrite the whole command again.
   - It's very tedious to run long pieces of code.

2. Script Mode
   (a) **啟動方式**

```
python filename.py
```

[8]Python Programming in Interactive vs Script Mode

(b) 優點:

- It is easy to run large pieces of code.
- Editing your script is easier in script mode.
- Good for both beginners and experts.

(c) 缺點:

- Can be tedious when you need to run only a single or a few lines of cod.
- You must create and save a file before executing your code.

2.4 安裝 Jupyter Notebook

1. Mac/Linux 系統

(a) 安裝 pip

i. 檢查是否已安裝 Python 套件管理工具 (命令提示字元)

```
python -m pip --version
pip --version
```

ii. 下載並安裝 pip(命令提示字元)

A. 下載 pip 安裝檔

點此下載 get-pip.py

B. 執行 get-pip.py

```
python get-pip.py
```

C. 升級 pip

```
pip3 install --upgrade pip
```

(b) 安裝 Jupyter(命令提示字元)

- 安裝

```
pip install jupyter
```

- 啟動

```
jupyter notebook
```

- 預設網址: `http://localhost:8888`

2. Windows 系統
   (a) 直接下載安裝Anaconda，內建 Jupyter
3. Jupyter Notebook 基本操作
   - 變更預設 port: XXXX

```
jupyter notebook --port XXXX
```

   - 登出後重新登入：tokenk Python

3 變數、資料型態、輸出輸入

3.1 Hello World

1. C++

```cpp
#include <iostream>
using namespace std;
int main() {
  cout << "Hellow world!";
}
```

2. Python

```python
print("Hello world!")
```

```python
print("This is a test")
```

```
This is a test
```

3.2 變數

1. 命名規則
   - 由英文、數字、底線、中文 (不建議) 組成
   - 不得以數字開頭
   - 不能與 Python 內建的保留字相同
2. 範例

Example	✓ / ×
abc_123	✓
3pigs	×
Happy New Year	×
Class	×
Good!	×

3. 變數的指派 (assign)
   - Python 變數不需宣告，依指派值自動設定資料型態 (dynamically typing)。
   - 語法

變 數 名 稱 = 指 派 值

- 變數不再使用時，可用 del 指令將其刪除，以節省記憶體。

- 範例：

```
a = 5
b = 3.14
c = 'TNFSH'
a = b = c = 10
name, number = 'TNFSH', 35
del c
print(name)
```

3.3 資料型態

1. 變數型態的判斷: type()

```
print(type(2020))
print(type(3.1416))
print(type('Hello world'))
```

2. 常用類型
   - 整數: int
   - 浮點數: float
   - 布林值 (True / False): bool, T 與 F 要大寫
   - 字串: str, 以' 或 ''含括, 若輸出字串要包含引號，則以另一種引號含括該字串。如:

```
age = 18
weight = 67.87
good_words = " 請 常 說 '請'、 '謝 謝'、 '對 不 起' "
```

   - 序列型態: list, tuple
   - 集合型態: set
   - 對映型態: dict

3. 型態間的轉換

- 自動轉換

```
score = 60 + 3.5                    # 自動轉換為浮點數，結果為63.5
```

- 強制轉換

```
score = int(30.22)                  # 將括弧內的資料轉換為整數
score = float(score)                # 將括弧內的資料轉換為浮點數
test = str(score)                   # 將括弧內的資料轉換為字串
```

3.4 進階輸出: print

1. 語法
   - print(項目 1, [項目 2, ⋯, sep = 分隔字元, end = 結束字元])
   - sep (分隔字元) 預設為空白字元
   - end (結束字元) 預設為換行字元
2. 範例

```
a, b = 5, 10
print(a, b)
print(a, b, sep =',')
print(a, b, sep ='分隔')
print(a, b, end='\r')
print(a, b, end='\r\n')
print(a, b, end='\t')
print(a, b, end='')
```

```
5 10
5,10
5 分隔 10
5 10
5 10
5 10 5 10
```

3. 跳脫字元 CRLF、CR、LF 詳解

Escape Sequence	Meaning
newline	Ignored
\\	Backslash (\)
\'	Single quote (')
\"	Double quote (")
\a	ASCII Bell (BEL)
\b	ASCII Backspace (BS)
\f	ASCII Formfeed (FF)
\n	ASCII Linefeed (LF)
\r	ASCII Carriage Return (CR)
\t	ASCII Horizontal Tab (TAB)
\v	ASCII Vertical Tab (VT)
ooo	ASCII character with octal value *ooo*
\x*hh*...	ASCII character with hex value *hh*...

Figure 8: Escape char

4. 格式化輸出: % (舊版)
   - 語法
     - 格式化輸出語法: print(字串%(參數))
     - 字串裡%s 代表字串、%d 代表整數、%f 代表浮點數
   - 範例

```
name = '台北101'
height = 508
fee = 18.24
print('%s的高度為%d公尺, 參觀門票為%8.2f美金.' %(name, height, fee))
print('%s的高度為%d公尺, 參觀門票為%0.2f美金.' %(name, height, fee))
```

```
台北 101 的高度為 508 公尺, 參觀門票為   18.24 美金.
台北 101 的高度為 508 公尺, 參觀門票為 18.24 美金.
```

5. 格式化輸出: format() (新版)

- 語法
  - format 格式化輸出語法：print(字串.format (參數))
  - 字串裡以 {0}、{1}、... 來對應參數列裡的變數
- 範例

```
name = 'Taipei101'
height = 508.35
print('{0}的高度為{1}公尺'.format (name, height))
print('{0:15s}的高度為{1:10.1f}公尺'.format (name, height))
print('{building}的高度為{height}'.format(building=name, height=height))
```

```
Taipei101 的高度為 508.35 公尺
Taipei101      的高度為     508.4 公尺
Taipei101 的高度為 508.35
```

6. 格式化輸出: Template

```
from string import Template

name = 'TNFSH'
tmp = Template('Hello, $who')
print(tmp.substitute(who = name))
```

```
Hello, TNFSH
```

7. 進階閱讀
   - 如何使用 Python 進行字串格式化

3.5 輸入

1. 語法/範例
   - 語法

```
variable = intpu([提示字元])
```

- PS: 經由 input( ) 函式讀入的資料，其資料型態皆為字串

- 範例

```
a = input('輸入國文成績:')
b = input('輸入數學成績:')
c = input()
print('三科成績分別為%5s %5s %5s' %(a, b, c))
```

2. 輸入搭配型別轉換

```
a = int(input('輸入國文成績:'))
b = int(input('輸入數學成績:'))
c = int(input())
print('總分為%5d' %(a+b+c))
```

3. input 注意事項
   - input 每筆輸入讀至換行符號為止

   - 若測試資料輸入以空白間隔，則讀入整行字串後，再以字串分割處理

```
inp = input()
inpstr = inp.split()
# 依此類推逐一取得輸入值
a = int(inpstr[0])
b = int(inpstr[1])
c = int(inpstr[2])
```

3.6 註解

- 語法

  - 單行註解：以 # 開頭

  - 多行註解：前後以"'' 或 "''" 含括

- 範例

```
## 這是單行註解
int a = 3
```

```
'''
這是多行註解
LALALA
'''
print(a)
"""
這也是
"""
```

4 運算元與運算式

4.1 算術運算

- +: 加
- -: 減
- *: 乘
- /: 除
- %: 取餘數
- //: 求商
- **: 指數

1. 算術運算範例 #1

```
# python code for arithematic opearation
print(5+3)
print(5-3)
print(5*3)
print(5/3)
print(5%3)
print(5//3)
print(5**3)
```

```
8
2
15
1.6666666666666667
2
1
125
```

2. 算術運算範例 #2

```
a = int(input("input a: "))
op = input("input op: ")
```

```
b = int(input("input b: "))

if op == '+':
    ans = a + b;
elif op == '-':
    ans = a - b;
elif op == '*':
    ans = a * b;
elif op == '/':
    ans = a / b;
elif op == '%':
    ans = a % b;

print(str(a) + op + str(b) + "=" + str(ans))
```

3. 字串運算

```
print("Python"+"基礎")
print(3*"Python基礎")
```

```
Python 基礎
Python 基礎 Python 基礎 Python 基礎
```

4.2 關係運算子

>	<	>=	<=	==	!=
大於	小於	大於等於	小於等於	等於	不等於

4.3 邏輯運算子

and	or	not
(a>b) and (a<c)	(a>b) or (a==b)	not (a>b)

4.4 複合指定運算子

+=	-=	*=	/=	%=	//=	**=

4.5 實作練習

- skyoj #161 華氏溫度轉攝氏溫度
- skyoj #277. 一元二次方程式求解

5 內建函數

5.1 基本函數與運算子

- ord(): 傳回某字元的 ASCII code /Unicode

```
print(ord('A'))
print(ord('©'))
```

```
65
169
```

- chr(): 傳回某 ASCII/Unicode 所代表的字元

```
print(chr(97))
print(chr(169))
```

```
a
©
```

- len(): 傳回字串長度

```
print(len('Python基礎'))
```

```
8
```

- max()/min(): 傳回字串 Unicode 最大/最小字元

```
print(max('Python基礎'))
print(min('Python基礎'))
```

```
礎
P
```

- str(): 將數值參數轉為字串

```
print(str(3.1416) + str(2020))
```

```
3.14162020
```

5.2 字串處理函數

1. 字串轉換函數
   - str.upper(): 將字串 s 所有字元轉成大寫

```
s = 'To be or not to BE'
print(s.upper())
```

   - str.lower(s): 將字串 s 所有字元轉成小寫

```
s = 'To be or not to BE'
print(s.lower())
```

```
to be or not to be
```

   - str.swapcase(s): 將字串 s 大小寫互換

```
s = 'To be or not to BE'
print(s.swapcase())
```

```
tO BE OR NOT TO be
```

   - str.capitalize(s): 首字大寫

```
s = 'To be or not to BE'
print(s.capitalize())
```

```
To be or not to be
```

   - str.title(s): 每個單字的第一個字元大寫

```
s = 'To be or not to BE'
print(s.title())
```

```
To Be Or Not To Be
```

   - str.replace(old, new): 置換子字串

```
s = 'To be or not to BE'
print(s.replace('be', 'die'))
```

```
To die or not to BE
```

2. **字串測試函數**
   - str.isupper(): **是否整段字串均為大寫英文字母**

```
s = 'To be or not to BE'
print(s.isupper())
print("PYTHON".isupper())
```

```
False
True
```

   - str.islower()

```
s = 'To be or not to BE'
print(s.islower())
print("python".islower())
```

```
False
True
```

   - str.isidentifier(): **是否為合法的識別字** (**含關鍵字**)

```
print("while".isidentifier())
print("computer".isidentifier())
print("__".isidentifier())
```

```
True
True
False
```

   - str.isalpha(): **字串是否只包含英文**

```
Print("Abcdxyz".Isalpha())
Print("1a2b".Isalpha())
```

```
True
False
```

- str.isdecimal(), str.isdigit(), str.isnumeric(): 字串是否為數字[9]以涵蓋範圍論，isdecimal() isdigit() isnumeric()，即，若一字串為 decimal，則一定也是 digit 和 numeric。

- isdigit()
  - True: Unicode 數字，byte 數字（單位元組），全形數字（雙位元組），羅馬數字
  - False: 漢字數字
  - Error: 無

- isdecimal()
  - True: Unicode 數字,，全形數字（雙位元組）
  - False: 羅馬數字，漢字數字
  - Error: byte 數字（單位元組）

- isnumeric()
  - True: Unicode 數字，全形數字（雙位元組），羅馬數字，漢字數字
  - False: 無
  - Error: byte 數字（單位元組）

- Some examples of characters isdecimal()==True [10]

  (a) "0123456789" DIGIT ZERO~NINE

  (b) "٠" ARABIC-INDIC DIGIT ZERO~NINE

  (c) "०" DEVANAGARI DIGIT ZERO~NINE

[9] python 判斷字串，str 函式 isdigit、isdecimal、isnumeric 的區別

[10] What's the difference between str.isdigit, isnumeric and isdecimal in python?

(d) “০” BENGALI DIGIT ZERO~NINE

(e) “০” GURMUKHI DIGIT ZERO~NINE

(f) “০” GUJARATI DIGIT ZERO~NINE

(g) “০” ORIYA DIGIT ZERO~NINE

(h) “০” TAMIL DIGIT ZERO~NINE

(i) “০” TELUGU DIGIT ZERO~NINE

(j) “০” KANNADA DIGIT ZERO~NINE

(k) “০” MALAYALAM DIGIT ZERO~NINE

(l) “০” THAI DIGIT ZERO~NINE

(m) “০” LAO DIGIT ZERO~NINE

(n) “০” TIBETAN DIGIT ZERO~NINE

(o) “০” MYANMAR DIGIT ZERO~NINE

(p) “০” KHMER DIGIT ZERO~NINE

(q) “０１２３４５６７８９” FULLWIDTH DIGIT ZERO~NINE

- Some examples of characters isdecimal()==False but isdigit()==True

(a) “źšş” SUPERSCRIPT ZERO~NINE

(b) “০” SUBSCRIPT ZERO~NINE

(c) “০” DIGIT ZERO~NINE FULL STOP

(d) “০” DIGIT ZERO~NINE COMMA

(e) “০” CIRCLED DIGIT ZERO~NINE

(f) “” NEGATIVE CIRCLED DIGIT ZERO~NINE

(g) “” PARENTHESIZED DIGIT ONE~NINE

(h) “” DINGBAT CIRCLED SANS-SERIF DIGIT ONE~NINE

(i) “” DOUBLE CIRCLED DIGIT ONE~NINE

(j) “” DINGBAT NEGATIVE CIRCLED SANS-SERIF DIGIT ONE~NINE

(k) “” ETHIOPIC DIGIT ONE~NINE

- Some examples of characters isdecimal()==False and isdigit()==False but isnumeric()==True

(a) “¡ij¿” VULGAR FRACTION

(b) “” BENGALI CURRENCY NUMERATOR

(c) “” TAMIL NUMBER TEN, ONE HUNDRED, ONE THOUSAND

(d) “” TELUGU FRACTION DIGIT

(e) “” MALAYALAM NUMBER, MALAYALAM FRACTION

(f) “” TIBETAN DIGIT HALF ZERO~NINE

(g) “” ETHIOPIC NUMBER TEN~NINETY, HUNDRED, TEN THOUSAND

(h) “” KHMER SYMBOL LEK ATTAK

(i) “” ROMAN NUMERAL

(j) “” SMALL ROMAN NUMERAL

(k) “” ROMAN NUMERAL

(l) “⊠⊠⊠

NUMBER TEN~FIFTY

(m) “⊠⊠⊠⊠⊠⊠⊠⊠” CIRCLED NUMBER TEN~EIGHTY ON BLACK SQUARE

(n) “⑳” PARENTHESIZED NUMBER TEN~TWENTY

(o) “㉑” NUMBER TEN~TWENTY FULL STOP

(p) “㉿” NEGATIVE CIRCLED NUMBER ELEVEN

(q) “➿” various styles of CIRCLED NUMBER TEN

(r) “➓” DINGBAT NEGATIVE CIRCLED SANS-SERIF DIGIT ZERO

(s) “〇” IDEOGRAPHIC NUMBER ZERO

(t) “〧〨〩〪〭〮〯〫〬〰〱〲” HANGZHOU NUMERAL ONE~TEN, TWENTY, THIRTY

(u) “⊠⊠⊠⊠” IDEOGRAPHIC ANNOTATION ONE~FOUR MARK

(v) “㈠㈡㈢㈣㈤㈥㈦㈧㈨㈩” PARENTHESIZED IDEOGRAPH ONE~TEN

(w) “⊠⊠⊠⊠⊠⊠⊠⊠⊠⊠” CIRCLED IDEOGRAPH ONE~TEN

(x) “一二三四五六七八九十壹貳參肆伍陸柒捌玖拾零百千萬億兆弌貳貳僅柒漆什乂陌阡佰仟万亿幺兩王三卄卅卌廿廿” CJK UNIFIED IDEOGRAPH

(y) “參拾兩零六陸什” CJK COMPATIBILITY IDEOGRAPH

```
print("4".isdecimal())
print("4".isdigit())
print("4".isnumeric())
print("⑳".isdecimal())
print("⑳".isdigit())
print("⑳".isnumeric())
print("四".isdecimal())
print("四".isdigit())
```

```
print("四".isnumeric())
```

```
True
True
True
False
False
True
False
False
True
```

- str.isspace(): 字串是否為空白

- str.istitle(): 字串是否每個單字

3. 搜尋子字串函數
   - str.count(s): 傳回字串中出現 s 的次數

```
x = "HIHI Python"
print(x.count('HI'))
```

```
2
```

- str.startswith(s): 字串是否以 s 開頭

```
x = 'HIHI Python'
print(x.startswith('HI'))
print(x.startswith('python'))
```

```
True
False
```

- str.endswith(s): 字串是否以 s 結尾

```
x = 'HIHI Python'
print(x.endswith('HI'))
print(x.endswith('python'))
```

```
False
False
```

- str.find(s): s 在字串中的位置 (找到的第一個)

```
x = 'HIHI Python'
print(x.find('HI'))
print(x.find('Python'))
print(x.find('python'))
```

```
0
5
-1
```

- str.rfind(s): s 在字串中的位置 (找到的最後一個)

```
x = 'HIHI Python'
print(x.rfind('HI'))
print(x.rfind('Python'))
print(x.rfind('python'))
```

```
2
5
-1
```

4. 切割/刪除字串
   - str.split([chars]): 從整個字串中根據 chars 來切割，若省略 chars 則以空白進行切割

```
x = 'To be or not to be.'
print(x.split())
print(x.split('be'))
```

```
['To', 'be', 'or', 'not', 'to', 'be.']
['To ', ' or not to ', '.']
```

   - str.strip([chars]): 由字串 * 兩側 * 刪除 chars，若省略 chars 則以空白進行切割

```
x = '  Cogito, ergo sum   '
print(x.strip())
print('ABCDEDCBA'.strip('A'))
```

```
Cogito, ergo sum
BCDEDCB
```

- str.lstrip([chars]): 由字串 * 左側 * 刪除 chars，若省略 chars 則以空白進行切割
- str.rstrip([chars]): 由字串 * 右側 * 刪除 chars，若省略 chars 則以空白進行切割

5. 格式化函數
   - str.center(width): 傳回欄位索度為 width 所指定的置中字串

```
s = '**'
x = 'Hello python'
print(s + x.center(20) + s)
```

```
**    Hello python    **
```

- str.ljust(width): 傳回欄位索度為 width 所指定的靠左字串

```
s = '**'
x = 'Hello python'
print(s + x.ljust(20) + s)
```

```
**Hello python        **
```

- str.rjust(width): 傳回欄位索度為 width 所指定的靠右字元

```
s = '**'
x = 'Hello python'
print(s + x.rjust(20) + s)
```

```
**        Hello python**
```

- str.zfill(width): 左側填零

```
s = '**'
x = 'Hello python'
print(s + x.zfill(20) + s)
```

```
**00000000Hello python**
```

- str.format(spec): 設定欄位寬度與對齊方式

```
s = '**'
x = 'Hello python'
print(s + format(x, '20') + s)
print(s + format(x, '>20') + s)
print(s + format(x, '<20') + s)
```

```
**Hello python        **
**        Hello python**
**Hello python        **
```

5.3 數值處理函數

- abs(x)
- min(x1, x2 [, x3...])
- max(x1, x2 [, x3...])
- pow(x, y)
- divmod(x, y)

5.4 作業

1. 小編日常
2. 小編日常二
3. 資安檢查

6 判斷結構

6.1 if

1. 語法
   - 條件式可不用括號 ( ) 含括，條件式後需搭配冒號：
   - 程式區塊以縮排方式處理，同一層縮排視為同一程式區塊

```
if condition:
    statement 1
    ...
```

2. 範例

```
num = 31
if num % 2 == 0:
    print('%d is even' %(num))
if num % 2 == 1:
    print('%d is odd' %(num))
```

```
31 is odd
```

6.2 if ... else ...

1. 語法

```
if condition:
    statement 1
    ...
else:
    statement 3
    ...
```

2. 範例

```
num = 32
if num % 2 == 0:
    print('%d is even' %(num))
else:
    print('%d is odd' %(num))
```

```
32 is even
```

6.3 if ... elif ... else ...

1. 語法

```
if condition 1:
    statement 1
    ...
elif condition 2:
    statement 3
```

```
        ...
    elif condition 3:
        statement 5
        ...
    else:
        statement N
```

2. 範例

```
score = 87
if score >= 90:
    print('A')
elif score >= 80:
    print('B')
elif score >= 70:
    print('C')
elif score >= 60:
    print('D')
else:
    print('F')
```

```
B
```

6.4 巢狀 if

1. 語法

```
if condition 1:
    statement 1
    ...
    if condition 2:
        statement 3
        ...
    else:
        statement 5
        ...
else:
    if conditi6on 3:
        statement 7
        ...
    else:
        statement N
```

16 ...

2. 範例: 某年份是否為閏年

```
digraph TLS{
  rankdir=LR;
  x1[shape=box, labeVisualStudioCodel = "輸入年份"];
  x2[shape=diamond, label = "4的倍數"];
  x3[shape=diamond, label = "100的倍數"];
  x4[shape=diamond, label = "400的倍數"];
  xY[shape=circle, fontcolor=Red, label="閏年"]
  xN[shape=circle, fontcolor=Blue, label="平年"]

  x1 -> x2;
  x2 -> xN[label="No"]; x2 -> x3[label="Yes"];
  x3 -> xY[label="No"]; x3 -> x4[label="Yes"];
  x4 -> xN[label="No"]; x4 -> xY[label="Yes"];
}
```

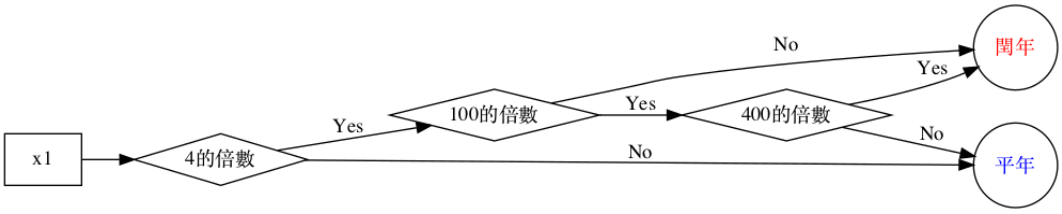


Figure 9: 閏年判斷流程

3. code #1

```
year = int(input("請輸入一個年份:"))
if (year % 4) == 0:
    if (year % 100) == 0:
        if (year % 400) == 0:
            print("%s年是世紀閏年" % year)
        else:
            print("%s年為平年" % year)
    else:
        print("%s年是普通閏年" % year)
else:
    print("%s年為平年" % year)
```

4. code #2

```
year = int(input("請輸入一個年份:"))
```

```
if (year % 4) == 0 and (year % 100) !=0 or (year % 400) == 0:
    print("%s年是閏年" % year)
else:
    print("%s年為平年" % year)
```

6.5 實作練習

- skyoj #162 成績等第判別
- skyoj #164 獎金給多少
- skyoj #278 三角關係

6.6 作業

1. 5D. No one is perfect

7 迴圈結構

7.1 for

1. 語法

```
for variable in sequence:
    statement 1
    ...
```

   - for 迴圈的變數會依序走訪 sequence 中的元素
   - sequence[11]可為 range 函式、字串 (string)、表列 (list)、元組 (tuple)、字典 (dict)、集合 (set)

2. List 為 range() function 語法

```
for variable in range([起始值,] 終止值 [,遞增值]):
    statement 1
    ...
```

   - 起始值預設為 0，遞增值預設為 1

[11]Python 數據類型-List 介紹

- 起始值 range( ) 的範圍 < 終止值

3. List 為 range() function 語法

```
for x in range(4):
    print(x)
print('========')
for x in range(3, 10):
    print(x, end=', ')
print('\n========')
for x in range(3, 10, 2):
    print(x, end=',')
```

```
0
1
2
3
========
3, 4, 5, 6, 7, 8, 9,
========
3,5,7,9,
```

4. List 為 String

```
school = 'TNFSH'
for x in school:
    print(x, end=' ')
print('\n=========')
for x in school:
    print(chr(ord(x)+1), end=' ')
```

```
T N F S H
=========
U O G T I
```

- ord( ) -> 將字元轉為 ASCII 編碼 (整數)

- chr( ) -> 將 ASCII 編碼 (整數) 轉為字元

```
school = '高雄市'
for x in school:
```

```
    print(x, end=' ')
print('\n=========')
for x in school:
    print(chr(ord(x)+1), end=' ')
```

```
高 雄 市
=========
高 雅 布
```

5. 課堂練習 以 FOR 輸出以下字串

```
abcdefghijklmnopqrstuvwxyz
zyxwvutsrqponmlkjihgfedcba
```

7.2 while

1. 語法

```
while (condition):
    statement 1
    ...
```

2. 範例

```
n = 12345
while (n > 0):
    print(n%10)
    n //= 10
```

```
5
4
3
2
1
```

```
m, n = 42, 75
while (n > 0):
    m, n = n, m % n
print(m)
```

```
3
```

7.3 break

跳出整個 loop

1. **範例**

```
for letter in 'Python':
    if letter == 'h':
        print('Now stop...')
        break
    print('Processing Letrer:', letter)
```

```
Processing Letrer: P
Processing Letrer: y
Processing Letrer: t
Now stop...
```

2. **練習** Given a positive integer N. The task is to write a Python program to check if the number is prime or not.

   - Definition:
     A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. The first few prime numbers are {2, 3, 5, 7, 11, ⋯.}.

   - Examples :

```
Input:  11
Output: true
Input:  15
Output: false
Input:  1
Output: false
```

7.4 continue

跳出單次 loop

1. **範例**

```
for letter in 'Python':
    if letter == 'h':
```

```
            print('Now skip...')
            continue
        print('Processing Letrer:', letter)
```

```
Processing Letrer: P
Processing Letrer: y
Processing Letrer: t
Now skip...
Processing Letrer: o
Processing Letrer: n
```

2. 練習 Given a string. Print all letters in this string except 'N' and 'S'.

   - Example:

```
Input: TNFSH
Output: TFH
```

7.5 for / while + else

Python 一個十分有趣的 loop 語法是它能配合 else 來用，語法如下：

```
for n in ...:
    if condition is true:
        ....
        break
else:
    print("all condition is false")
```

如果上述的 for 迴圈從頭到尾都沒有去跑過 break(也就是說 condition 都不成立，for 正常結束)，那麼 else 裡的程式碼就會被執行。那麼，何時會用到這個有趣的語法呢？

以質數判斷為例：原本的寫法為:

```
n = 22
isPrime = True
for x in range(2, n//2 + 1):
    if n % x == 0:
        isPrime = False
```

```
        print("Composite Number!")
        break
if isPrime:
    print("Prime number")
```

```
Composite Number!
```

套用 else 後則可改為

```
n = 23
for x in range(2, n//2 + 1):
    if n % x == 0:
        print("Composite Number!")
        break
else:
    print("Prime number")
```

```
Prime number
```

當我們要在 for/while 內佈署一些條件判斷 (如果條件成立就 break，例如，判斷某數是否為質數)，如果找到因數，則臟

1. reading: [Python] Loop 配合 else 的妙用

7.6 巢狀迴圈

1. 範例

```
for i in range(1, 10):
    for j in range(1, 10):
        print('%d*%d=%2d' %(i, j, i*j), end=' ')
    print()
```

```
1*1= 1 1*2= 2 1*3= 3 1*4= 4 1*5= 5 1*6= 6 1*7= 7 1*8= 8 1*9= 9
2*1= 2 2*2= 4 2*3= 6 2*4= 8 2*5=10 2*6=12 2*7=14 2*8=16 2*9=18
3*1= 3 3*2= 6 3*3= 9 3*4=12 3*5=15 3*6=18 3*7=21 3*8=24 3*9=27
4*1= 4 4*2= 8 4*3=12 4*4=16 4*5=20 4*6=24 4*7=28 4*8=32 4*9=36
5*1= 5 5*2=10 5*3=15 5*4=20 5*5=25 5*6=30 5*7=35 5*8=40 5*9=45
6*1= 6 6*2=12 6*3=18 6*4=24 6*5=30 6*6=36 6*7=42 6*8=48 6*9=54
7*1= 7 7*2=14 7*3=21 7*4=28 7*5=35 7*6=42 7*7=49 7*8=56 7*9=63
8*1= 8 8*2=16 8*3=24 8*4=32 8*5=40 8*6=48 8*7=56 8*8=64 8*9=72
9*1= 9 9*2=18 9*3=27 9*4=36 9*5=45 9*6=54 9*7=63 9*8=72 9*9=81
```

7.7 實作練習

1. 所有位數平方和 輸入一整整數，輸出該整數所有位數平方和
   Example

   ```
   Input: 12345
   Output: 55
   ```

2. 求 1+2+3+...+n<=m，已知 m 時之最大的 n 值。輸入 m，輸出 n。
3. 輸出 99 乘法表
4. 拉斯維加斯

7.8 作業

1. 5A. 因數和
2. 5C. 所有位數和
3. 6B. 保安〜〜可以讓人這樣一乘再乘嗎?
   - 輸入：讀入一整數 n、接下來讀入 n 個整數，最後輸入一個整數 m。
   - 輸出：將所有 n 個數相乘、再將所得乘積除以 m、輸出餘數
   - 提示：(a * b) % c = ((a % c) * (b % c)) % c

8 資料型別

8.1 字串 string

1. 資料格式 以單引號 (')d [index]: 字串中 index 所在字元
   - [start:end:increment]: 截取部份字串
   - in: 判斷是否為子字串
2. 範例 2

```
tel = '06-2371206'
ext = '#600'
# +
print('tel+ext:',tel+ext)
# *
```

```
print('ext*2:', ext*2)
# [index]
print('tel[5]:', tel[5])
# [start:end:increment]
print('tel[1:4]:',tel[1:4])
print('tel[6:␣]:',tel[6: ])
print('tel[␣:6]:',tel[ :6])
print('tel[::-1]:',tel[::-1])
# in
print("'9'␣in␣tel:", '9' in tel)
```

```
tel+ext: 06-2371206#600
ext*2: #600#600
tel[5]: 7
tel[1:4]: 6-2
tel[6: ]: 1206
tel[ :6]: 06-237
tel[::-1]: 6021732-60
'9' in tel: False
```

3. 字串 function 與 method
   - len(<str>): 計算字串長度
   - <str>.lower(): 字串轉小寫
   - <str>.upper(): 字串轉大寫
   - <str>.islower(): 字串中英文全大寫
   - <str>.isupper(): 字串中英文全小寫
   - <str>.find(<str1>): 在 <str> 尋找 <str1>，回傳索引值；若未找到，回傳-1
   - <str>.replace(<str1>, <str2>): 將 <str> 中的 <str1> 以 <str2> 取代
   - <str>.split([sep]): 字串以 sep 分割, sep 預設值為空白
4. 範例 3

```
school = 'Tnfsh'
```

```
print('school:', school)
print('len(school):', len(school))
print('school.lower():', school.lower())
print('school.isupper():', school.isupper())
print("school.find('fsh'):", school.find('fsh'))
print("school.replace('fsh', ssh'):", school.replace('fsh', 'ssh'))
print("school.split('f'):", school.split('f'))
```

```
school: Tnfsh
len(school): 5
school.lower(): tnfsh
school.isupper(): False
school.find('fsh'): 2
school.replace('fsh', ssh'): Tnssh
school.split('f'): ['Tn', 'sh']
```

8.2 列表 list

1. 資料格式
   - 以 [ ] 將不同型態的資料含括起來，以, 分隔
   - 表列中的資料是有序排列，從 0 開始編號
   - 格式：表列名稱 = [元素 1, 元素 2, ⋯]

```
data = ['John', [95, 118], 'May', 100]
print(data[0])
print(data[1])
print(data[1][1])
print(data[2])
print(data[2][1])
```

```
John
[95, 118]
118
May
a
```

2. list 函式與方法

- len(<list>): 計算表列元素個數
- list(<str>): 將 <str> 轉成表列
- <list>.clear(): 清除表列中所有元素
- <list>.append(<obj>): 將 <obj> 加到 <list> 尾端
- <list>.extend(<list1>): 將 <list1> 合併至 <list> 尾端
- <list>.remove(<obj>): 移除 <list> 中 <obj> 元素
- <list>.insert(<i>,<obj>): 將 <obj> 加到 <list> 的索引值 <i> 位置
- <list>.pop([index]): 從 <list> 取出指定元素 若 [index] 未指定，則取出尾端元素
- <list>.reverse(): 反轉表列元素
- sum(<list>): 將表列中所有元素加總 (註：僅限表列元素皆為數字)
- <list>.sort(): 將表列中所有元素排序
- '[sep]'.join(<list>): 用 [sep] 來連結表列元素，若未指定 [sep]，則無間隔 (僅限表列元素皆為字串)

3. list 範例

```
data = ['John', [95, 118], 'May', 100]
print(len(data))
print(len(data[1]))
print(list('TNFSH'))
data.clear()  #清空list
data.append(35)
print(data)
data1 = [89, 'James', 100]
data.extend(data1)
print(data)
data.remove('James')
print(data)
```

```
data.insert(1, 200)
print(data)
data.pop()
data.reverse()
print(data)
print(sum(data))
data.sort()
print(data)
# string
data_s = ["a", "b", "c"]
sep = "-"
print(sep.join(data_s))
```

```
4
2
['T', 'N', 'F', 'S', 'H']
[]
[35]
[35, 89, 'James', 100]
[35, 89, 100]
[35, 200, 89, 100]
[89, 200, 35]
324
[35, 89, 200]
a-b-c
```

4. **課堂練習 將以下** list **複製貼上至程式碼中**

```
scores = [78, 12, 97, 45, 12, 45, 98, 100, 9, 23]
```

輸出以下資訊

- **全班平均：至小數點第** 2 **位**
- **全班最高分分數**
- **全班最低分分數**

5. zip()

(a) zip 與 for 迴圈 在 Python 中若要將兩個 list 以迴圈的方式一次各取一個元素出來處理，可以使用 zip 打包之後配合 for 迴圈來處理，如下例

```
# 第一個 List
names = ["A", "B", "C"]

# 第二個 List
values = [11, 23, 46]

# 使用 zip 同時迭代兩個 List
for x, y in zip(names, values):
    print(x, y)
```

這裡的 zip(names, values) 會將 names 與 values 的每個元素以一對一的方式配對起來，組成一個新的迭代器，然後交給 for 迴圈進行迭代，所以每一次迭代時所取的 x 值會來自於 names，而 y 則會來自於 values[12]。結果如下：

```
A 11
B 23
C 46
```

(b) 不同長度的 Lists zip 若遇到不同長度的 list 時，會以長度最短的 list 為準，超過長度的部分就會被捨棄。zip 也可應用於兩個以上的 list，如：

```
# 多個 Lists
names = ["A", "B", "C"]
values = [11, 23, 46]
ages = [45, 67, 82]

# 使用 zip 同時迭代多個 List
for x, y, z in zip(names, values, ages):
    print(x, y, z)
```

6. list 練習
   - 連續輸入成績 (輸入-1 結束)，將成績由高至低排序輸出，並輸出總分、平均

```
score = []
while(True):
```

[12]Python 使用 zip 與 for 迴圈同時對多個 List 進行迭代

```
    sc = int(input())
    if (sc != -1):
        score.append(sc)
    else:
        break
score.sort(reverse=True)
print(score)
sos = sum(score)
print('全班總分: {0:d},全班平均: {1:0.2f}'.format(sos, sos/len(score)))
```

- 連續輸入成績 (加入提示字元)

```
sum = 0
grades = []
for i in range(0, 5):
    x = int(input("請輸入第" + str(i + 1) + "個成績:␣"))
    grades.append(x)
    sum += x
print(sum/5)
print(grades)
```

7. 課堂練習
   - Given a two Python list. Iterate both lists simultaneously such that list1 should display item in original order and list2 in reverse order
   - Example:

```
Input: [可以將輸入直接複製貼上到 code 中]
list1 = [10, 20, 30, 40]
list2 = [100, 200, 300, 400]
Output:
10 400
20 300
30 200
40 100
```

8. 作業
   (a) lone wolf 讀入一整數 n、接下來讀入 n 個整數，列出這幾個整數中未成對者

- Examples

```
Input: 3 2 2 1
Output: 1
Input: 5 4 1 2 1 2
Output: 4
```

(b) removing the Nth occurence element

- 讀入一字串
- 再讀入一 word x, 一整數 n (n< 字串長度)，刪除上述字串中第 n 個重複出現的 x
- 輸出剩下的字串
- Examples

```
Input: to be or not to be, that is the question.
       be
       2
Output: to be or not to, that is the question.
```

(c) Maximum Subarray item description here...

9. 2d List

(a) 練習: 利用 for-loop 取出二維 List 中的元素

```
ClassNo = [[101,102,103], [201,202,203], [301,302,303]]
```

8.3 元組 tuple

A tuple in Python is similar to a list. The difference between the two is that we cannot change the elements of a tuple once it is assigned whereas we can change the elements of a list.[13]

1. 資料格式
   - 資料格式：以 ( ) 將不同型態的資料含括起來，以, 分隔
   - 元組中的資料是有序排列，從 0 開始編號

[13]Python Tuple

- **格式：元組名稱** = (**元素** 1, **元素** 2, ⋯)
- **與表列** list **類似，但元組** tuple **內的元素不能修改**

2. Creating a Tuple A tuple is created by placing all the items (elements) inside parentheses (), separated by commas. The parentheses are optional, however, it is a good practice to use them.

```
# Different types of tuples

# Empty tuple
my_tuple = ()
print(my_tuple)

# Tuple having integers
my_tuple = (1, 2, 3)
print(my_tuple)

# tuple with mixed datatypes
my_tuple = (1, "Hello", 3.4)
print(my_tuple)

# nested tuple
my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))
print(my_tuple)

tup1 = (53, )     #只有一個元素時，其後要加逗號
print(tup1)
```

```
()
(1, 2, 3)
(1, 'Hello', 3.4)
('mouse', [8, 4, 6], (1, 2, 3))
```

3. Access Tuple Elements
   (a) Indexing We can use the index operator [] to access an item in a tuple, where the index starts from 0.

```
# Accessing tuple elements using indexing
```

```
my_tuple = ('p','e','r','m','i','t')

print(my_tuple[0])    # 'p'
print(my_tuple[5])    # 't'

# IndexError: list index out of range
# print(my_tuple[6])

# Index must be an integer
# TypeError: list indices must be integers, not float
# my_tuple[2.0]

# nested tuple
n_tuple = ("mouse", [8, 4, 6], (1, 2, 3))

# nested index
print(n_tuple[0][3])        # 's'
print(n_tuple[1][1])        # 4
```

```
p
t
s
4
```

(b) Negative Indexing Python allows negative indexing for its sequences.

```
# Negative indexing for accessing tuple elements
my_tuple = ('p', 'e', 'r', 'm', 'i', 't')

# Output: 't'
print(my_tuple[-1])

# Output: 'p'
print(my_tuple[-6])
```

```
t
p
```

(c) Slicing We can access a range of items in a tuple by using the slicing operator colon :.

```
# Accessing tuple elements using slicing
my_tuple = ('p','r','o','g','r','a','m','i','z')

# elements 2nd to 4th
# Output: ('r', 'o', 'g')
print(my_tuple[1:4])

# elements beginning to 2nd
# Output: ('p', 'r')
print(my_tuple[:-7])

# elements 8th to end
# Output: ('i', 'z')
print(my_tuple[7:])

# elements beginning to end
# Output: ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
print(my_tuple[:])
```

```
('r', 'o', 'g')
('p', 'r')
('i', 'z')
('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
```

(d) Tuple Methods Methods that add items or remove items are not available with tuple. Only the following two methods are available.

```
my_tuple = ('a', 'p', 'p', 'l', 'e',)

print(my_tuple.count('p'))  # Output: 2
print(my_tuple.index('l'))  # Output: 3
```

```
2
3
```

(e) Tuple 遍歷

```
# Using a for loop to iterate through a tuple
for name in ('John', 'Kate'):
    print("Hello", name)
```

```
Hello John
Hello Kate
```

4. 練習

(a) Modify the first item (maybe any number, 33 in this case) of a list inside a following tuple to n Examples

```
tuple1 = (11, [22, 33], 44, 55)
Expected output:
tuple1 = (11, [222, 33], 44, 55)
```

```
tuple1 = (11, [22, 33], 44, 55)
tuple1[1][0] = 222
print(tuple1)
```

```
(11, [222, 33], 44, 55)
```

(b) 練習: 史上最強掌法 自從鐵掌無敵馬掌門出任武林盟主後，綠林豪傑一片哀鴻，民不聊生，痛苦指數破表，眾家名門弟子甚至遠避異鄉以求溫飽。近來馬掌門更獨創「油電雙掌」，以圖鞏固武林領導地位，此套掌法高深莫測，武林中人聞風喪膽，少林、武當各派高手不得不拋開門戶之見，齊聚「竹園崗」合作商議對策。目前僅由歷來幾次掌下逃生者的對戰經驗分析出以下數據

與馬掌門對戰時間 (秒)	對戰者每秒接受之傷害值
120 以下	2.10
121~330	3.02
331~500	4.39
501~700	4.97
701 以上	5.63

請你幫這些可憐的高手寫一個程式分析對戰時間與受傷指數間的關係。

輸入：0 ~ 10000 秒之間任意值

輸出：受傷指數

i. 解 1

```
#sec = input()
sec = 800
if sec <= 120:
    print(sec*2.10)
elif sec > 120 and sec <= 330:
    print(120*2.10+(sec-120)*3.02)
```

```
elif sec > 330 and sec <= 500:
    print(120*2.10+(330-120)*3.02+(sec-300)*4.39)
elif sec > 500 and sec <= 700:
    print(120*2.10+(330-120)*3.02+(500-330)*4.39+(sec
        -500)*4.97)
elif sec > 700:
    print(120*2.10+(330-120)*3.02+(500-330)
        *4.39+(700-500)*4.97+(sec-700)*5.63)
```

```
3189.5
```

ii. 解 2

```
#sec = input()
sec = 800
hurt = 0
gap = (700, 500, 330, 120, 0)
rate = (5.63, 4.97, 4.39, 3.02, 2.10)
for i in range(5):
    if sec > gap[i]:
        hurt += (sec-gap[i])*rate[i]
        sec = gap[i]
print(hurt)
```

```
3189.5
```

8.4 字典 dict

1. 資料格式
   - 以 { } 將各組鍵: 值對應資料含括起來，以, 分隔
   - 字典中的資料是無序的
   - 格式：字典名稱 = {k1:v1, k2:v2, ⋯}
   - 範例：
   - 註：若字典中有相同的 key，則會取出最後的 value
2. 範例

```
data = { 'John': 95, 'May': 100 }
print(data['May'])
```

```
100
```

3. dic 操作
   - 新增、修改、刪除

```
data = { 'John': 95, 'May': 100 }
# 新增
data['Harrison'] = 88
# 修改
data['John'] = 99
# 刪除鍵值對
del data['John']
print(data)
# 刪除dic
del data
# print(data) --> NameError: name 'data' is not defined
```

```
{'May': 100, 'Harrison': 88}
```

4. dic method
   (a) 增/剛
      - <dict>.clear()
      - <dict>.popitem()
      - <dict>.copy()

```
data = { 'John': 95, 'May': 100, 'John': 105 }
data1 = data.copy()
print("data1 = data.copy():",data1.copy())
print("data:",data)
print("data.clear():",data.clear())
pop1 = data1.popitem()
print(pop1)
```

```
data1 = data.copy(): {'John': 105, 'May': 100}
data: {'John': 105, 'May': 100}
data.clear(): None
('May', 100)
```

(b) 取值

- <dict>.get(key, default=None)
- <dict>.items()
- <dict>.keys()
- <dict>.values()

```
data = { 'Vanessa': 95, 'May': 100, 'John': 105 }
print("data.get('John'):",data.get('John'))
print("data.items():",data.items())
print("data.keys():",data.keys())
print("data.values(),data.values"())
```

```
data.get('John'): 105
data.items(): dict_items([('Vanessa', 95), ('May', 100), ('John', 105)])
data.keys(): dict_keys(['Vanessa', 'May', 'John'])
data.values() dict_values([95, 100, 105])
```

(c) fromkes

- <dict>.fromkeys(<seq>[, val]): Python 字典 fromkeys() 函數用於創建一個新字典，以序列 seq 中元素做字典的鍵，value 為字典所有鍵對應的初始值。

```
name = ('Vanessa', 'May', 'John')
data = dict.fromkeys(name)
print(data)
data = dict.fromkeys(name, 100)
print(data)
```

```
{'Vanessa': None, 'May': None, 'John': None}
{'Vanessa': 100, 'May': 100, 'John': 100}
```

(d) 遍歷 Dict

```
data = { 'John': 95, 'May': 100, 'Vanessa': 105, 'James':
    999 }
for k, v in data.items():
    print(k, " : ", v)
```

```
John  :  95
May  :  100
Vanessa  :  105
James  :  999
```

5. **練習** Get the key corresponding to the minimum value from the following dictionary

```
sampleDict = {
  'Physics': 82,
  'Math': 65,
  'history': 75
}

Expected output:
Math
```

8.5 List v.s. Tuple

二者的差異與適用時機[3]**:**

1. **能否改變內容**
   - Using a tuple instead of a list can give the programmer and the interpreter a hint that the data should not be changed.

2. **可讀性**
   - Reading data is simpler when tuples are stored inside a list. For example,

```
[(2,4), (5,7), (3,8), (5,9)]
```

   is easier to read than

```
[[2,4], [5,7], [3,8], [5,9]]
```

3. Tuple **與** dictionary **的相似性**
   - Tuples are commonly used as the equivalent of a dictionary without keys to store data. For Example,

```
[('Swordfish', 'Dominic Sena', 2001), ('Snowden', ' Oliver Stone', 2016), ('Taxi Driver', 'Martin Scorsese', 1976)]
```

- Above example contains tuples inside list which has a list of movies.

- Tuple can also be used as key in dictionary due to their hashable and immutable nature whereas Lists are not used as key in a dictionary because list can't handle __hash__() and have mutable nature.

```
key_val= {('alpha','bravo'):123} #Valid
#key_val = {['alpha','bravo']:123} #Invalid
print(key_val)
print(key_val.keys())
print(key_val.values())
```

```
{('alpha', 'bravo'): 123}
dict_keys([('alpha', 'bravo')])
dict_values([123])
```

8.6 集合 set

1. 資料格式
   - 資料格式：以 {} 將各組資料含括起來，以',' 分隔, 或以 set() 建立

   - 集合中的資料是無序的，會自動刪除重複元素

   - 格式

```
集合名稱 = {元素1, 元素2, … }
```

   - 註：set(<seq>) 函式的參數 <seq> 可為字串、表列、元組、字典

2. 空集合 如果你要建立空集合，不可以直接使用 {} 不包括任何元素的實字寫法，因為 {} 不包括任何元素的實字寫法表示一個空字典物件, 如果你要建立空集合, 必須使用 set 類別建構[14]。例如：

```
family = set()
family.add("James")
family.add("Vanessa")
print(family)
```

```
{'James', 'Vanessa'}
```

[14]集合

3. 範例

```
S1 = { 'John', 95, 'May', 100, 'John' }
S2 = set('apple')    {'l', 'a', 'e', 'p'}  # 集合的資料是無序的
```

4. 元素新增刪除

```
<set>.add(<item>)
<set>.remove(<item>)
```

- remove() 若無此 item 會發生錯誤

```
s = set('apple')
s.add('x')
print(s)
s.remove('p')
print(s)
```

```
{'e', 'p', 'x', 'l', 'a'}
{'e', 'x', 'l', 'a'}
```

5. 集合可使用函式 與串列 (List) 和數組 (Tuple) 一樣可以使用以下函式[15]

- len(): 回傳長度
- sum(): 回傳總和
- max(): 回傳最大值
- min(): 回傳最小值

6. 判斷某 element 是否存在於 set 中 與串列 (List) 和數組 (Tuple) 一樣可以使用 in 和 not in 來判斷元素是否存在於集合中

```
set1 = {2, 4, 6, 8, 10}

print(2 in set1)
print(11 in set1)
print(3 not in set1)
print(4 not in set1)
```

[15] [Python] 學習使用集合 (Set)

```
True
False
True
False
```

7. 遍歷 set 因為集合 (Set) 沒辦法使用索引 (Index) 來印出, 所以用 for 迴圈寫時要這樣寫

```
set1 = {2, 4, 6, 8, 10}

for i in set1:
        print(i, end = ' ')
```

```
2 4 6 8 10
```

8. 集合運算

運算	符號
聯集	|
交集	&
差集	-
互斥	^
屬於	in

```
a = {'a', '2', '3', '4'}
b = set('357')
print("a | b:", a | b)
print("a & b:", a & b)
print("a - b:", a - b)
print("a ^ b:", a ^ b)
print("'5' in a:", '5' in a)
print("'5' in b:", '5' in b)
```

```
a | b: {'a', '4', '3', '2', '5', '7'}
a & b: {'3'}
a - b: {'a', '4', '2'}
a ^ b: {'a', '2', '5', '4', '7'}
'5' in a: False
'5' in b: True
```

9. 練習 Remove items from set1 that are not common to both set1 and set2

```
set1 = {10, 20, 30, 40, 50}
set2 = {30, 40, 50, 60, 70}

Expected output:
{40, 50, 30}
```

8.7 作業 [set]

1. 測資讀取的方式
   - OJ 上如何讀測資的教學?
   - What's the equivalent for while (cin » var) in python?

2. 你快樂嗎? 我很快樂 快樂數有以下的特性：在給定的進位制下，該數字所有數位（英語：digits）的平方和，得到的新數再次求所有數位的平方和，如此重複進行，最終結果必為 1。[16]
   - 輸入任一整數，回傳 Happy/Unhappy 來說明這個數是否快樂。
   - Examples:

```
Input: 28
Output: Happy
Input: 4
Output: Unhappy
```

3. read data into Dict dict 輸入購物清單，轉成 dict，輸出 dict，輸出總金額
   週年慶到了，將敗家視為日常活動的小珍開始認真研究各家百貨公司寄來的敗家專用廣告。由於小珍是百貨公司敗家排行榜中的 VVIP，自然在啟動敗家專案前自然是要先做好規劃。
   以下這列字串是小珍隨手寫下的敗家行動計劃，請你幫忙把這份計劃轉成較為清楚的格式，並列出全部金額以及最高金額的項目名稱。

4. unique character in string check set 讀入字串若干行 (我也不知道有幾行)，每行代表一集合。輸出所有集合的交集項目數與聯集項目數。
   A,B,D,X,Q,M,G,C
   B,C,Y,N,M,L,H
   Q,C,Y,M,D,H

[16] 快樂數

5. word frequency dict 讀入一段字，輸出所有字的頻率

8.8 資料型別整理

資料型別	符號	資料可改?	資料有序?	資料類型
字串 string	' ' 或 " "			相同 (字元)
表列 list	[ ]			可不同
元組 tuple	( )			可不同
字典 dict	{ }			相同 (鍵: 值)
集合 set	{ }			可不同

9 FUNCTION

9.1 What for?

1. 作業 5A: No one is perfect

```
import sys
a , b = int(input()), int(input())
d=0
for i in range(a,b+1):
    c=0
    for j in range(1,i):
        if i%j==0:
            c=c+j
    if c == i and c!=1:
        print(i)
        sys.exit()
print("No number is perfect")
```

2. 如果有個 fucntion 叫 isPerfect, 傳入一整數, 若該整數為完全數, 則傳回 True, 否則傳回 False, 那上述 solution 就能改為:

```
import sys
a , b = int(input()), int(input())
for i in range(a,b+1):
    c=0
    if isPerfect(i):
        print(i)
        sys.exit()
print("No number is perfect")
```

9.2 函式 (function) 基本操作

1. 函式 (function) 的重要性
   - function 為一段程式的集合，可視為一個獨立的區段。
   - function 可重覆使用，因此為結構化程式語言的重要元素。可將龐大複雜的程式分解為小問題，由多人分別以 function 解決，縮短開發時間。
   - function 可區分為以下三類：

  - Python 內建 function，如: print()、len()、int()、str()...。
  - 第三方公司所開發的模組 module(多個 function 組合)。
  - 自定義 function。

2. 自定義函式
   - 宣告語法

```
def 函式名稱 ( [參數1, 參數2, ...] ):
    程式區塊
    [ return 值 ]
```

   - 說明:
     - 參數可省略，亦即呼叫 function 不需傳入任何資料 (引數)。
     - 若無回傳值，則無需 return。
     - 若多個回傳值可用逗號隔開。

3. 自定義函式的呼叫
   - 呼叫函式語法:

```
[ 變數 ] = 函式名稱( [引數1, 引數2, ... ] )
```

```
def circle(r):
    return r*r*3.14;

r = 10
print('半徑%d的圓面積: %.2f' %(r, circle(r)))
```

```
半徑 10 的圓面積: 314.00
```

4. 多參數函式
   - 呼叫函式時引數 (arugment) 個數
   - 自定義函式的參數 (parameter) 個數

- 設定參數初始值。若呼叫時無引數傳入，則使用其初始值。

```
def printInfo(name, school='TNFSH'):
    print('學生:',name,'就讀:',school)
    return

printInfo(name='James')
printInfo(name='Harrison',school='TNSSH')
```

```
學生: James 就讀: TNFSH
學生: Harrison 就讀: TNSSH
```

5. 多回傳值函式
   - 呼叫函式多個回傳值語法

```
[變數1, 變數2, ... ] = 函式名稱( [引數1, 引數2, ... ] )
```

```
def scoreProcessor(score):
    sos = 0
    for x in score:
        sos += x
    # sos = sum(score)
    return sos, sos/len(score)

scores = (21, 33, 83, 100, 75, 60)
sum, avg = scoreProcessor(scores)
print('sum=',sum,',average=',avg)
```

```
sum= 372 ,average= 62.0
```

9.3 引數 (argument) 的傳遞

- 每一個函式都是獨立的，所以函式只知道自己程式區域內的變數，不認識函式外的變數。
- 變數因其所宣告位置，可區分為全域變數 (Global variable)、區域變數 (Local variable)。
  - 全域變數：宣告在任何函式外，所有函式皆可存取。
  - 區域變數：宣告在函式內，僅函式內可存取。c

- **全域變數與區域變數範例**

```
def printGlobal(x, y):
    print(a+b)

def printLocal(a, b):
    print(a+b)

a, b = 5, 10
printGlobal(10, 10)
printLocal(10,10)
```

```
15
20
```

1. **傳值與傳址**
   - **引數** (argument) **的傳遞分為兩種方式，由** Python **自動判別：**
     - **傳值呼叫** (Call-by-Value)
     - **傳址呼叫** (Call-by-Reference)

引數傳遞	傳遞內容	參數改變	資料型別
傳值呼叫 call-by-value	引數值	否	bool, int, float, str, tuple, chr
傳址呼叫 call-by-reference	引數記憶體位址	是	list, dict, set

```
# str 傳值、list 傳址
def ccc(name, score):
    name = 'Amy'
    score.append(99)
    print('——函式內輸出——')
    print('name:', name,'\tscore:',score)

name = 'Josh'
score = [77, 88]

ccc(name, score)
print('——主程式輸出——')
print('name:', name,'\tscore:',score)
```

```
--函式內輸出---
name: Amy  score: [77, 88, 99]
--主程式輸出--
name: Josh  score: [77, 88, 99]
```

9.4 模組 (module)

1. **匯入模組語法：** A library in Python is called a module.

```
import 模組1 [, 模組2, ... ]
from 模組 import 函式
```

2. **範例**

```
import random as ran

print(ran.randint(1, 100))
print(ran.randint(1, 100))
print(ran.randint(1, 100))
```

```
50
78
16
```

3. **自訂模組**
   - **定義**: myModule.py

```
# name: myModule.py

def pp(n):
    return n+1

def mm(n):
    return n-1
```

   - **使用**

```
import myModule as my

print(my.pp(5))
print(my.mm(5))
```

```
print(my.__name__)
print(dir(my))
```

```
6
4
myModule
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__packag
```

9.5 遞迴 (recursive)

1. 語法
   - 遞迴 (recursive)：一個函式直接呼叫函式本身

```
def fib(n):
    if n == 1 or n == 2:
        return 1
    else:
        return fib(n-1) + fib(n-2)

print('The %d-th Fibonacci number = %d' %(10, fib(10)))
```

```
The 10-th Fibonacci number = 55
```

2. 基本條件
   - 遞迴程式設計要滿足兩條件:

     (a) 遞迴關係式，如

```
fib(n) = fib(n-1) + fib(n-2)
sum(n) = n + sum(n-1)
```

     (b) 遞迴終止式，如

```
f(1) = f(2) = 1
sum(0) = 0
```

   - 範例:GCD-while

```
m, n = 20, 50
while (n>0):
```

```
        m, n = n, m % n
    print(m)
```

```
10
```

3. 範例
   - 範例:GCD-recursive

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

a, b = 10, 50
print(gcd(a, b))
```

```
10
```

9.6 實作練習

1. 計算 n! 值。輸入 n，輸出 n! = XXXXX。如輸入 5，輸出 5! = 120。
2. 計算 C(n, r) 組合數。輸入 n, r，輸出 C(n, r) = XX。如輸入 9, 4，輸出 C(9, 4) = 126。
3. Harmonic sumRecursion Write a Python program to calculate the harmonic sum of n-1. Note: The harmonic sum is the sum of reciprocals of the positive integers.

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + ... + \frac{1}{n-1}$$

4. Sum of recursive list list:recursion Write a Python program of recursion list sum. Go to the editor
   Test Data: [1, 2, [3,4], [5,6]]
   Expected Result: 21